Datasheet for the Dataset: Tyrol Avalanche Danger Level and Snow/Weather Features

Von Dominik Glätzle and Malte Braig

Project: Data Analysis Project, Foundations of Data Engineering and Analytics **Version / date:** v0.4, 2026-10-23
Research question: How accurately can the avalanche warning level issued by LWD Tirol be predicted based on weather and snow data from previous days, and which factors (fresh snow, temperature, wind, snow depth) are key to this prediction?

---

1. Motivation

For what purpose was the dataset created? To predict the daily avalanche danger level (1–5) per Tyrolean warning region from snow and weather conditions of the preceding days. We found no dataset links LWD Tirol bulletins to station weather/snow data in one table, so we built it.

---

2. Composition

What do the instances represent? One instance = one **(date, warning region)** pair. Each row holds the danger level issued for that date/region plus weather and snow features. Region IDs are `AT-07-01` … `AT-07-29`, plus iner sub-region IDs introduced later (e.g. `AT-07-02-01`).(Region IDs are **not stable across the full date range**).

How many instances are there? 36178 rows (`src/data_engineering_project/data/combined.csv`), from 2019-11-16 to 2026-05-01, across **47 distinct region codes** (29 "base" codes in the earliest seasons, more after later splits).

Is it all possible instances or a sample? A sample: 7 winters (`2019/20`–`2025/26`) from the Tyrolean bulletin archive. Season start/end dates vary year to year (roughly late Nov to early May):

Season	First date	Last date	Rows
2019/20	2019-11-16	2020-05-03	4929
2020/21	2020-12-04	2021-05-09	4553
2021/22	2021-12-01	2022-05-01	4408
2022/23	2022-12-10	2023-05-01	5148
2023/24	2023-11-25	2024-05-01	5724
2024/25	2024-12-07	2025-05-01	5256
2025/26	2025-11-29	2026-05-01	6160

Representativeness untested. 7 winters won't cover all snowpack types and weather situations.

What data does each instance consist of? (35 columns)

Keys and bulletin information

Column	Type	Description
`date`	date	Day the bulletin is valid for.
`season`	string	Winter season, e.g. `2019/20`.
`region`	string	Warning region ID.
`region_name`	string	Region name.
`status`	string	Only value present: `ok` (100%).
`danger_max`	int	`= max(danger_above, danger_below, danger_am, danger_pm)` exactly, no exceptions. **Target**
`danger_above`	int	Danger level above the elevation limit.
`danger_below`	float	Danger level below the limit.
`elev_limit`	string	always non-empty whenever `danger_above != danger_below`
`danger_am`, `danger_pm`	int	AM/PM danger level, not always equal to each other or to `danger_above`
`problems`	string	`;`-separated EAWS problem types. Vocabulary: `wind_slab`, `persistent_weak_layers`, `gliding_snow`, `wet_snow`, `new_snow`, `favourable_situation`.
`danger_patterns`	string	`;`-separated EAWS danger-pattern codes `DP1`–`DP10`, all ten occur.
`tendency`	string	`steady`, `increasing`, `decreasing` or empty.
`published`	timestamp	Bulletin publish time (UTC).
`n_bulletins`	int	1 (36145 rows) or 2 (33 rows). Merge rule (`combine.py: dedupe_bulletins`): later `published` time wins, same-time duplicates merge via max danger level + union of `problems`/`danger_patterns`/`tendency`.

Weather and snow features — units from `parameter_metadata.csv` / `preprocessing.ipynb`'s `RENAME` dict:

Column	Type	Description
`air_temp_mean/min/max`	float	°C (param `tl`).
`wind_speed_mean`, `wind_gust_max`	float	m/s (params `ff`/`ffx`).
`wind_dir_deg`	float	°, 0–360. A vector mean (sin/cos components averaged, then `atan2`), not a dominant sector.
`rel_humidity_mean`	float	% (param `rf`).
`pressure_mean`	float	hPa, **station-level**, not sea-level-reduced (param `p`).
`sunshine_sum`	float	hours (param `so_h`).
`snow_depth_cm`	float	cm (param `sh`).
`dist_snow_station_km`, `dist_precip_station_km`	float	Distance to the nearest of the 3 selected stations for that variable, 0–93.4 km.

Column	Type	Description
`precip_sum_mm`	float	mm (param `rr`).
`snow_delta_24h_cm`, `snow_delta_72h_cm`	float	`snow_depth_cm.diff(1)` / `.diff(3)` per region.
`precip_sum_3d_mm`, `precip_sum_7d_mm`	float	3-/7-day rolling sums per region.
`air_temp_mean_3d`, `wind_gust_max_3d`	float	3-day rolling mean / max per region.

No leakage, by design. `combine.py`'s `daily_station_features()` buckets each hourly reading into a bulletin date so the 24h window for row `date = d` ends at 17:00 local on `d-1` — exactly the bulletin's publish cutoff. So every weather/snow value is something the forecaster could already see.

Station → region assignment (`assign_regions.py` + `combine.py: region_weather`): per region and per variable, the 3 nearest stations with ≥30% day-coverage are combined with inverse-distance weights `1/(km+1)`. This is why `dist_*_station_km` can reach 93 km — some regions have no close station with usable data for a given variable.

[TODO: station elevation is in `stations_metadata.csv` aber nich in `combined.csv` absicht?]

Is any information missing from individual instances? See missing_values.ipynb

Are relationships between instances explicit? Rows share keys `date`/`region`, are ordered in time, and the rolling features overlap between consecutive days -> rows are not independent.

Recommended data splits? [TODO: Brauchen wird des jetzt schon?]

Errors, sources of noise, redundancies?

- **Station-to-region mismatch**: assigned stations can be up to 93.4 km from a region.
- **Redundant columns**: `danger_max` is fully derivable from the other 4 danger columns.
- **Region IDs are not stable.** `AT-07-02/04/14/17` were split into finer sub-regions starting season 2022/23, and `AT-07-23/26/29` starting 2025/26 — this is why the region count grows from 29 to 47. Any per-region time split must account for this. Minor: `region_name` for `AT-07-01` changes mid-archive, and `AT-07-02`'s name contains non-breaking-space characters.
- Danger levels are expert judgement, which includes subjective variation and real experiments on the Mountain.

Is the dataset self-contained? No, built from external sources e.g. raw downloads from archives.

Source	Used for	Access
LWD Tirol / EUREGIO avalanche bulletins	danger levels, problems, patterns, tendency	CAAML v6 JSON, `lawinen.report/bulletins/{date}/{date}_EUREGIO_de_CAAMLv6.json`
GeoSphere Austria	temperature, wind,	GeoSphere Data Hub API

Source	Used for	Access
station data, `klima-v2-1h`	humidity, pressure, sunshine, precipitation, snow depth	

---

3. Collection Process

How was the data acquired?

- **Danger levels:** produced by LWD Tirol forecasters, as CAAML v6 JSON (`avalanche_euregio.py`).
- **Weather/snow:** observed hourly at GeoSphere stations (`klima-v2-1h`), aggregated to station-day then region-day then rolling windows (`combine.py`).

Mechanisms/procedures used for collection? Scripts in `src/data_engineering_project/`: `avalanche_euregio.py` downloads/parses bulletins (cached, retried), station data (`station_data.csv`, `stations_metadata.csv`) was pulled from the GeoSphere Data Hub by a step not in the repo, then cleaned in `preprocessing.ipynb` (drops `substation`/`tb100`, renames columns), `assign_regions.py` assigns stations to regions, `combine.py` builds the final table.

Sampling strategy? Deterministic: all bulletin days within the 7 selected winters.

Who was involved? Dominik Glätzle, Malte Braig

Over what timeframe was the data collected? 22.10.2026 , data covers seasons 2019/20–2025/26.

Ethical review? Not applicable (no human subjects / personal data).

---

4. Preprocessing / Cleaning / Labeling

- Bulletins reduced to one row per (`date`, `region`), `danger_max = max(above, below, am, pm)`, `elev_limit` empty if `above == below`.
- Station values assigned to regions via 3-nearest, inverse-distance weighting with a 30% coverage filter.
- Derived: 24h/72h snow-depth deltas, 3-/7-day precip sums, 3-day temp mean, 3-day gust max.
- No unit conversions, units kept sames as GeoSphere.
- Leakage control: the 24h feature window ends exactly at the bulletin's publish cutoff (17:00 local)
- Rolling-window start-of-season (TODO: how do we calc this ???)

Was the raw data saved? Yes, in `src/data_engineering_project/data/` alongside the derived `combined.csv`.

Is the preprocessing software available? Yes: `preprocessing.ipynb`, `avalanche_euregio.py`, `assign_regions.py`, `combine.py` ...

---

5. Uses

Has the dataset been used already? Only in this course project.

Other possible tasks: personal Projects

Anything that might affect future use?

- The target is a human forecast, not an observed avalanche, a model learns to imitate forecasters, not predict avalanches.
- 7 winters is a small, possibly unrepresentative sample.
- Point station data + large regions limit how precisely features describe where danger arises.

It should not be used for: real avalanche safety decisions, use official bulletins and local assessment.

---

6. Distribution

Distributed to third parties? Only to course staff via OLAT.

Third-party restrictions / export controls: None known.

---

7. Maintenance

Who maintains it? The project team until course end (final submission 2027-01-19).

Sources

- EUREGIO avalanche bulletins (CAAML v6 JSON): `lawinen.report/bulletins/{date}/{date}_EUREGIO_de_CAAMLv6.json`
- GeoSphere Austria station data, `klima-v2-1h`: https://dataset.api.hub.geosphere.at/app/frontend/station/historical/klima-v2-1h
- Gebru et al., Datasheets for Datasets: https://arxiv.org/abs/1803.09010